SENAI

PROGRAMAÇÃO BACK-END 2

Introdução ao Node.js e configuração do ambiente

Digitar, executar, observar e registrar

Projeto api-produtos | Aula 01

Missão da aula

A dinâmica será sempre: digitar, executar, observar e concluir

- Preparar o ambiente e executar JavaScript pelo terminal
- Usar scripts npm para repetir comandos do projeto
- Ler argumentos e configurações do processo
- Distinguir sucesso e falha pelo código de saída
- Transformar api-produtos em um diagnóstico reproduzível

Ao final, você será capaz de

- Executar arquivos com Node.js
- Reconhecer recursos do runtime e do navegador
- Usar process.argv e process.env
- Criar e executar scripts npm
- Devolver códigos de saída coerentes

Fundamentos essenciais

F

Compreender o runtime antes de observar seus efeitos no Prompt de Comando.

JavaScript fora do navegador

JavaScript

A linguagem escrita nos arquivos .js

Node.js

O runtime que cria e executa o processo

Sistema operacional

Arquivos, processos, rede e terminal

Motor V8: quem executa o código

Código JavaScript

Instruções escritas pela aplicação

Motor V8

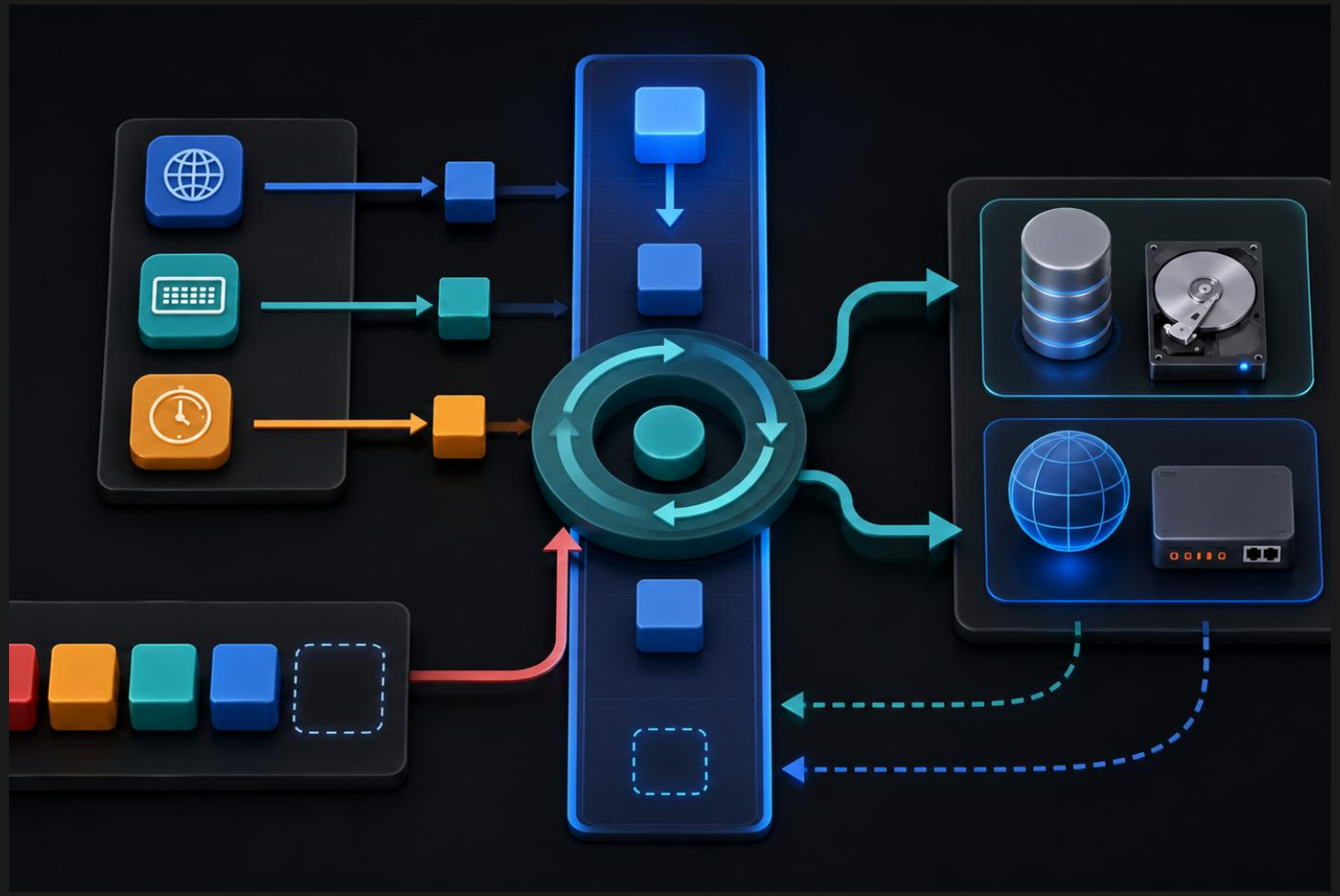
Interpreta e compila o JavaScript

Node.js

Integra o V8 com recursos do sistema

Single-thread e Event Loop

Uma thread principal executa o JavaScript; o Event Loop coordena o retorno das tarefas.



1. O JavaScript percorre a thread principal.
2. Operações demoradas podem ser delegadas.
3. O Event Loop coordena a retomada.

I/O bloqueante e não bloqueante

Fluxo bloqueante

- Inicia a operação
- Espera disco ou rede
- Só então continua

Fluxo não bloqueante

- Delega a espera
- Continua outras tarefas
- Retoma com o resultado

Fundamentos: verifique a compreensão

Responda antes de executar o próximo comando

Qual afirmação descreve melhor o Node.js?

A) Uma nova linguagem que substitui JavaScript

B) Um runtime que executa JavaScript fora da página

C) Um servidor web iniciado por qualquer arquivo .js

Mapa da aula

- Preparação do Node.js e do Sandbox
- Ambiente de execução
- Sucesso e falha no terminal
- Argumentos e scripts npm
- Configuração pelo ambiente
- Evolução prática da api-produtos

00

Confirme as ferramentas e crie um espaço permanente para os testes rápidos.

Confirme Node.js e npm

O Node.js deve começar com v24.

```
node --version
npm --version
```

Crie o Sandbox permanente

Os testes rápidos e suas saídas ficarão registrados aqui.

```
mkdir exemplos\Exemplo00-Sandbox
cd exemplos\Exemplo00-Sandbox
```

Mini-desafio 1

01

Descobrir qual ambiente está executando o JavaScript.

Execute as duas expressões

```
node -e "console.log('Executado pelo Node.js')"
node -e "console.log(document.title)"
```

Por que somente a segunda falhou?

Responda antes de executar o próximo comando

Qual explicação combina com a evidência?

A) JavaScript só funciona no navegador

B) document pertence ao ambiente de uma página

C) Node.js não reconhece console.log

Linguagem e ambientes

Node.js

- Cria um processo
- Fornece process
- Executa e termina
- Não abre página por padrão

Navegador

- Executa JavaScript
- Fornece document
- Fornece window
- Interage com páginas

Crie o arquivo (runtime.js)

```
console.log({
  node: process.version,
  plataforma: process.platform,
  arquitetura: process.arch,
  diretorio: process.cwd(),
});
```

Execute e compare

Compare diretorio com o caminho mostrado no prompt do CMD.

```
node runtime.js
```

Mini-desafio 2

02

Usar o código de saída para comunicar sucesso e falha.

Observe dois processos

```
node -e "console.log('Tudo certo')"
echo %ERRORLEVEL% OU echo $LASTEXITCODE

node -e "console.error('Falha controlada'); process.exitCode = 1"
echo %ERRORLEVEL% OU echo $LASTEXITCODE
```

O contrato de um processo

Código 0

- Execução concluída
- Resultado esperado
- Automação pode continuar

Código diferente de 0

- Falha comunicada
- Resultado incompleto
- Automação pode interromper

Texto vermelho é suficiente?

Responda antes de executar o próximo comando

console.error foi executado, mas o código final é 0. O que falta?

A) process.exitCode = 1

B) console.log(process.env)

C) npm init -y

Teste rápido em runtime.js

Confirme o código 2 e depois remova a linha temporária.

```
process.exitCode = 2;

node runtime.js
echo %ERRORLEVEL% OU echo $LASTEXITCODE
```

Mini-desafio 3

03

Receber dados digitados depois do nome do arquivo.

Crie o arquivo (argumentos.js)

```
const [, , nome, turma] = process.argv;

if (!nome || !turma) {
  console.error('Uso: node argumentos.js <nome> <turma>');
  process.exitCode = 1;
} else {
  console.log({ nome, turma });
}
```

Execute os dois caminhos

node argumentos.js Marina 3DS-A
node argumentos.js
echo %ERRORLEVEL% OU echo $LASTEXITCODE

Como process.argv organiza os dados

- Índice 0: caminho do executável Node.js
- Índice 1: caminho do arquivo executado
- Índice 2: primeiro argumento digitado
- Índice 3: segundo argumento digitado
- Todos os argumentos chegam como texto

Mini-desafio 4

04

Registrar comandos do projeto com nomes estáveis.

Inicialize o manifesto

Permaneça dentro de Exemplo00-Sandbox.

```
npm init -y
```

Configure private e scripts

```
{
  "private": true,
  "scripts": {
    "start": "node argumentos.js",
    "check": "node --check argumentos.js"
  }
}
```

Investigue os scripts

```
npm run
npm run check
npm start -- Caio 3DS-B
npm run comando-inexistente
```

Checagem e execução

node --check

- Valida a sintaxe
- Não executa o programa
- Não testa entradas

Teste funcional

- Executa o comportamento
- Percorre sucesso e falha
- Confirma o código final

Manifesto e lockfile

- package.json descreve o projeto e seus scripts
- package-lock.json registra a instalação reproduzível
- npm ci exige coerência entre os dois arquivos
- Nos projetos fornecidos, prefira npm ci

Mini-desafio 5

05

Ler configuração sem gravar valores no código.

Crie o arquivo (ambiente.js)

```
const obrigatorias = ['NOME_ALUNO', 'TURMA'];
const ausentes = obrigatorias.filter(
  (nome) => !process.env[nome]?.trim()
);

if (ausentes.length) {
  console.error(`Configure: ${ausentes.join(', ')}`);
  process.exitCode = 1;
} else {
  console.log({ estudante: process.env.NOME_ALUNO,
    turma: process.env.TURMA });
}
```

Primeiro, execute sem valores

```
node ambiente.js
echo %ERRORLEVEL% OU echo $LASTEXITCODE
```

Depois, configure o terminal atual

```
set NOME_ALUNO=Ana OU $env:NOME_ALUNO="Ana"
set TURMA=3DS-A OU $env:TURMA="3DS-A"
node ambiente.js
```

process.env com responsabilidade

- Entrega texto ou undefined
- Valores vazios também devem ser recusados
- Mostre somente as configurações necessárias
- Nunca imprima todo o ambiente
- Não revele senha, token ou outro segredo

Documente as chaves esperadas

Conteúdo de .env.example

```
PORT=3000
NOME_ALUNO=Seu nome
TURMA=3SEM
```

Crie .env no Sandbox

Salve como .env ao lado de ambiente.js

```
NOME_ALUNO=Ana
TURMA=3DS-A
```

Carregue a cópia local (ambiente.js)

Insira a primeira linha no topo de ambiente.js, salve e execute

```
process.loadEnvFile('.env');

node ambiente.js
```

Teste válido e inválido

Responda antes de executar o próximo comando

O que deve mudar quando uma variável obrigatória fica vazia?

A) Somente a cor do texto

B) A mensagem e o código de saída

C) A versão do Node.js

Evolução hands-on da api-produtos

06

Construa a primeira versão do projeto do zero, arquivo por arquivo.

Passo 1: crie um projeto limpo

Saia do Sandbox e inicie a API sem arquivos fornecidos

```
cd ..\..
mkdir api-produtos
cd api-produtos
npm init -y
```

Passo 2: Altere o manifesto (package.json)

Substitua o conteúdo de package.json | parte 1 de 2

```
{
  "name": "api-produtos",
  "version": "1.0.0",
  "description": "",
  "main": "app.js",
  "scripts": {
    "start": "node app.js",
    "diagnostico": "node app.js",
    "check": "node --check app.js"
  },
```

Finalize o manifesto

package.json | parte 2 de 2

```
  "keywords": [],
  "author": "",
  "license": "ISC",
  "private": true
}
```

app.js | carregar configuração

Trecho 1 de 4

```
const arquivoDeConfiguracao = process.argv[2];
let configuracaoCarregada = true;

if (arquivoDeConfiguracao) {
  try {
    process.loadEnvFile(arquivoDeConfiguracao);
  } catch {
    console.error(`Arquivo de configuração não encontrado: ${arquivoDeConfiguracao}`);
    process.exitCode = 1;
    configuracaoCarregada = false;
  }
}
```

app.js | localizar ausentes

Trecho 2 de 4

```
const obrigatorias = ['PORT', 'NOME_ALUNO', 'TURMA'];
const ausentes = [];

for (const nome of obrigatorias) {
  const valor = process.env[nome];

  if (typeof valor !== 'string' || valor.trim() === '') {
    ausentes.push(nome);
  }
}
```

app.js | tratar validação

Trecho 3 de 4

```
if (configuracaoCarregada && ausentes.length > 0) {
  console.error(`Configure no .env: ${ausentes.join(', ')}`);
  process.exitCode = 1;
} else if (configuracaoCarregada) {
  console.table({
    estudante: process.env.NOME_ALUNO,
    turma: process.env.TURMA,
    projeto: 'api-produtos',
    ambiente: process.env.NODE_ENV || 'development',
```

app.js | concluir diagnóstico

Trecho 4 de 4

```
    node: process.version,
    sistema: `${process.platform} ${process.arch}`,
    diretorio: process.cwd(),
    portaConfigurada: process.env.PORT
  });
  console.log('Ambiente configurado com sucesso.');
}
```

Por que usar try / catch?

Sem proteção

- Arquivo .env não existe
- loadEnvFile lança ENOENT
- Node.js encerra com exceção

Com proteção

- catch intercepta o erro
- Mensagem explica a causa
- exitCode registra falha 1

Passo 5: crie .env.example

Conteúdo de .env.example

```
PORT=3000
NOME_ALUNO=Seu nome
TURMA=3SEM
```

Passo 6: simule o sucesso

A tabela deve terminar com: Ambiente configurado com sucesso.

```
npm run check
copy .env.example .env
npm start
```

Confirme a tabela de ambiente

- Estudante e turma
- Projeto e ambiente
- Versão do Node.js
- Sistema e arquitetura
- Diretório atual
- Porta configurada

Passo 7: provoque a falha

```
del .env
npm run diagnostico
echo %ERRORLEVEL% OU echo $LASTEXITCODE
```

Falha por variáveis ausentes

Configure no .env: PORT, NOME_ALUNO, TURMA

Código de saída esperado: 1

Confirme também o try / catch

Sem .env, o catch deve produzir a mensagem controlada e o código 1

```
npm start
echo %ERRORLEVEL% OU echo $LASTEXITCODE
```

Falha por arquivo inexistente

Arquivo de configuração não encontrado: .env

Código de saída esperado: 1

Passo 8: relacione scripts e resultados

- start: tenta carregar o arquivo .env
- diagnostico: valida sem indicar arquivo
- check: valida a sintaxe sem executar
- Explique por que as duas falhas retornam 1

Diagnóstico orientado por evidências

07

Use o terminal para localizar a causa antes de alterar o código.

Sintoma e primeira ação

Sintoma	Evidência ou correção
node não reconhecido	where node
package.json ausente	node -p "process.cwd()"
.env ausente	copy .env.example .env
Erro com código 0	Definir exitCode = 1

Quando node --check passa

- A sintaxe está válida
- A lógica ainda pode estar errada
- Execute o caminho de sucesso
- Execute o caminho de falha
- Compare mensagem, dados e código de saída

08

Cinco aplicações independentes para consolidar o diagnóstico.

Atividades 01 a 05

- 01: completar plataforma e diretório
- 02: adicionar o script start ao painel
- 03: validar nome e turma em argumentos
- 04: localizar configurações ausentes
- 05: combinar chamado e ambiente sem revelar token

Protocolo de trabalho

- Leia o README da pasta
- Execute o estado inicial
- Implemente somente o TODO
- Repita o caminho válido
- Repita o caminho inválido
- Registre a evidência no Sandbox

Fechamento prático

09

Reproduza a sequência completa sem consultar o roteiro.

Sequência final

```
node --version
npm run check
copy .env.example .env
npm start
del .env
npm run diagnostico
echo %ERRORLEVEL% OU echo $LASTEXITCODE
```

Checklist de saída

- Node.js 24 responde no Prompt de Comando
- Scripts npm executam comandos previsíveis
- Argumentos chegam por process.argv
- Configurações chegam por process.env
- Sucesso retorna 0 e falha retorna valor diferente de 0
- O Sandbox preserva scripts e resultados

Síntese da aula

- JavaScript é a linguagem; Node.js é o runtime
- O terminal inicia e observa o processo
- process reúne runtime, argumentos e ambiente
- npm organiza comandos e instalação
- api-produtos termina como diagnóstico reproduzível

Referências oficiais

Documentação usada na Aula 01

Node.js: primeiros passos	nodejs.org/en/learn/getting-started/introduction-to-nodejs
Node.js: process	nodejs.org/api/process.html
Node.js: linha de comando	nodejs.org/api/cli.html
npm: package.json	docs.npmjs.com/cli/v11/configuring-npm/package-json